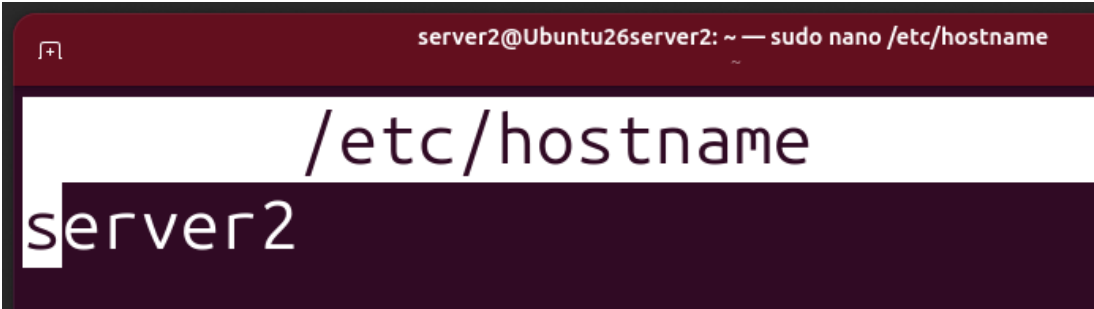


COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	07-09-26	DATE SUBMITTED:	07-13-26
NAME:	Santiago, David Owen A.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 1	Creating Virtual Machines
1. PROCEDURE OUTPUT	
Task 1: Do the following on Server 1, Server 2, and Local Machine. In editing the file using nano command, press control + O to write out (save the file). Press enter when asked for the name of the file. Press control + X to end.	
1. Change the hostname using the command <i>sudo nano /etc/hostname</i>	
1.1 Use server1 for Server 1	
	
1.2 Use server2 for Server 2	
	
1.3 Use workstation for the Local Machine	

```
santiago@Ubuntu26: ~ — sudo nano /etc/hostname

GNU nano 8.7.1 /etc/hostname
workstation
```

2. Edit the hosts using the command `sudo nano /etc/hosts`. Edit the second line.

2.1 Type 127.0.0.1 server 1 for Server 1

```
server1@Ubuntu26server1: ~ — sudo nano /etc/hosts

GNU nano 8.7.1 /etc/hosts *
# 127.0.0.1 localhost.localdomain localhost
::1 localhost6.localdomain6 localhost6
127.0.0.1 server1
```

2.2 Type 127.0.0.1 server 2 for Server 2

```
server2@Ubuntu26server2: ~ — sudo nano /etc/hosts

GNU nano 8.7.1 /etc/hosts *
#127.0.0.1 localhost.localdomain localhost
::1 localhost6.localdomain6 localhost6
127.0.0.1 server2
# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
```

2.3 Type 127.0.0.1 workstation for the Local Machine

```
santiago@Ubuntu26: ~ — sudo nano /etc/hosts
GNU nano 8.7.1 /etc/hosts *
# 127.0.0.1 localhost.localdomain localhost
::1 localhost6.localdomain6 localhost6
127.0.0.1 workstation

# The following lines are desirable for IPv6 capability>
::1 localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Task 2: Configure SSH on Server 1, Server 2, and Local Machine. Do the following:

1. Upgrade the packages by issuing the command *sudo apt update* and *sudo apt upgrade* respectively.

```
Get:115 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libglib2.0-0
4 amd64 2.80.0-6ubuntu3.8 [1545 kB]
Get:116 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 netplan-genera
tor amd64 1.1.2-8ubuntu1~24.04.2 [61.2 kB]
Get:117 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-netpla
n amd64 1.1.2-8ubuntu1~24.04.2 [24.3 kB]
Get:118 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 initramfs-tool
s-core all 0.142ubuntu25.8 [50.5 kB]
Get:119 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 amd64-microcod
e amd64 3.20251202.1ubuntu0.24.04.1 [458 kB]
Get:120 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-firmware
amd64 20240318.git3b128b60-0ubuntu2.27 [641 MB]
34% [120 linux-firmware 444 MB/641 MB 69%] 2058 kB/s 7min 24s
```

```
Installing new version of config file /etc/bsd-release ...
motd-news.service is a disabled or a static unit not running, not starting it.
(Reading database ... 212024 files and directories currently installed.)
Preparing to unpack .../bsdutils_1%3a2.39.3-9ubuntu6.5_amd64.deb ...
Unpacking bsdutils (1:2.39.3-9ubuntu6.5) over (1:2.39.3-9ubuntu6.2) ...
Setting up bsdutils (1:2.39.3-9ubuntu6.5) ...
(Reading database ... 212024 files and directories currently installed.)
Preparing to unpack .../coreutils_9.4-3ubuntu6.2_amd64.deb ...
Unpacking coreutils (9.4-3ubuntu6.2) over (9.4-3ubuntu6) ...

Progress: [ 1%] [.....]
```

2. Install the SSH server using the command *sudo apt install openssh-server*.

```
santiago@Ubuntu26:~$ sudo apt install openssh-server
Installing:
  openssh-server
```

Installing dependencies:

```
ncurses-term  openssh-sftp-server  ssh-import-id
```

Suggested packages:

```
molly-guard  monkeysphere  ssh-askpass
```

Summary:

```
Upgrading: 0, Installing: 4, Removing: 0, Not Upgrading: 3
Download size: 949 kB
```

```
server1@Ubuntu26server1:~$ sudo apt install openssh-server -y
Installing:
  openssh-server
```

Installing dependencies:

```
ncurses-term  openssh-sftp-server  ssh-import-id
```

Suggested packages:

```
molly-guard  monkeysphere  ssh-askpass
```

Summary:

```
Upgrading: 0, Installing: 4, Removing: 0, Not Upgrading: 3
Download size: 949 kB
```

```
server2@Ubuntu26server2:~$ sudo apt install openssh-server -y
Installing:
  openssh-server

Installing dependencies:
  ncurses-term  openssh-sftp-server  ssh-import-id

Suggested packages:
  molly-guard  monkeysphere  ssh-askpass

Summary:
  Upgrading: 0, Installing: 4, Removing: 0, Not Upgrading: 76
  Download size: 949 kB
  Space needed: 7,581 kB / 22.1 GB available
```

3. Verify if the SSH service has started by issuing the following commands:

3.1 *sudo service ssh start*

3.2 *sudo systemctl status ssh*

```
santiago@Ubuntu26:~$ sudo service ssh start && sudo systemctl status ssh
[sudo: authenticate] Password:
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Mon 2026-07-13 08:02:31 UTC; 23ms ago
   Invocation: 422af894a11c4602a7076ba498e701e5
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 7408 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 7411 (sshd)
    Tasks: 1 (limit: 4088)
   Memory: 1.3M (peak: 2.2M)
      CPU: 21ms
   CGroup: /system.slice/ssh.service
           └─7411 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jul 13 08:02:30 Ubuntu26 systemd[1]: Starting ssh.service - OpenBSD Secure Shell
```

```
server1@Ubuntu26server1:~$ sudo service ssh start && sudo systemctl status ssh
[sudo: authenticate] Password:
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Mon 2026-07-13 08:04:01 UTC; 36s ago
  Invocation: 1a137b7bef324c0b80617a57ec7757af
 TriggeredBy: ● ssh.socket
    Docs: man:sshd(8)
          man:sshd_config(5)
   Process: 5667 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 5669 (sshd)
    Tasks: 1 (limit: 6436)
   Memory: 1.2M (peak: 1.9M)
      CPU: 21ms
   CGroup: /system.slice/ssh.service
           └─5669 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jul 13 08:04:01 Ubuntu26server1 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Jul 13 08:04:01 Ubuntu26server1 sshd[5669]: Server listening on 0.0.0.0 port 22.
Jul 13 08:04:01 Ubuntu26server1 sshd[5669]: Server listening on :: port 22.
Jul 13 08:04:01 Ubuntu26server1 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
server1@Ubuntu26server1:~$
```

```
server2@Ubuntu26server2:~$ sudo service ssh start && sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Mon 2026-07-13 08:05:13 UTC; 14s ago
  Invocation: 4236cf7513b7457da5d6299198f27a38
 TriggeredBy: ● ssh.socket
    Docs: man:sshd(8)
          man:sshd_config(5)
   Process: 8252 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 8254 (sshd)
    Tasks: 1 (limit: 4088)
   Memory: 1.2M (peak: 1.9M)
      CPU: 23ms
   CGroup: /system.slice/ssh.service
           └─8254 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jul 13 08:05:13 Ubuntu26server2 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Jul 13 08:05:13 Ubuntu26server2 sshd[8254]: Server listening on 0.0.0.0 port 22.
Jul 13 08:05:13 Ubuntu26server2 sshd[8254]: Server listening on :: port 22.
Jul 13 08:05:13 Ubuntu26server2 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
server2@Ubuntu26server2:~$
```

SSH is active but disabled. Issuing the command “`sudo systemctl enable ssh`” can be used to enable it.

4. Configure the firewall to all port 22 by issuing the following commands:

4.1 `sudo ufw allow ssh`

4.2 `sudo ufw enable`

4.3 `sudo ufw status`

```
santiago@Ubuntu26:~$ sudo ufw allow ssh && sudo ufw enable && sudo ufw status
Rules updated
Rules updated (v6)
Firewall is active and enabled on system startup
Status: active
```

To	Action	From
--	-----	----
22/tcp	ALLOW	Anywhere
22/tcp (v6)	ALLOW	Anywhere (v6)

```
santiago@Ubuntu26:~$
```

```
server1@Ubuntu26server1:~$ sudo ufw allow ssh && sudo ufw enable && sudo ufw status
Rules updated
Rules updated (v6)
Firewall is active and enabled on system startup
Status: active
```

To	Action	From
--	-----	----
22/tcp	ALLOW	Anywhere
22/tcp (v6)	ALLOW	Anywhere (v6)

```
server1@Ubuntu26server1:~$
```

```
server2@Ubuntu26server2:~$ sudo ufw allow ssh && sudo ufw enable && sudo ufw status
Rules updated
Rules updated (v6)
Firewall is active and enabled on system startup
Status: active
```

To	Action	From
--	-----	----
22/tcp	ALLOW	Anywhere
22/tcp (v6)	ALLOW	Anywhere (v6)

```
server2@Ubuntu26server2:~$
```

Task 3: Verify network settings on Server 1, Server 2, and Local Machine. On each device, do the following:

- Record the ip address of Server 1, Server 2, and Local Machine. Issue the command *ifconfig* and check network settings. Note that the ip addresses of all the machines are in this network 192.168.56.XX.
 - Workstation IP address: 10.0.2.15

```
santiago@Ubuntu26:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fd17:625c:f037:2:a00:27ff:feb2:721c prefixlen 64 scopeid 0x0<global>
    inet6 fe80::a00:27ff:feb2:721c prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:b2:72:1c txqueuelen 1000 (Ethernet)
    RX packets 727729 bytes 1076895035 (1.0 GB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 36909 bytes 2311409 (2.3 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

1.2 Server 1 IP address: 10.0.2.15

```
server1@Ubuntu26server1:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fd17:625c:f037:2:a00:27ff:fe72:84d9 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::a00:27ff:fe72:84d9 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:72:84:d9 txqueuelen 1000 (Ethernet)
    RX packets 4209 bytes 1904027 (1.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 4385 bytes 2893688 (2.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

1.3 Server 2 IP address: 10.0.2.15

```
server2@Ubuntu26server2:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fe80::a00:27ff:fe52:9654 prefixlen 64 scopeid 0x20<link>
    inet6 fd17:625c:f037:2:a00:27ff:fe52:9654 prefixlen 64 scopeid 0x0<global>
    ether 08:00:27:52:96:54 txqueuelen 1000 (Ethernet)
    RX packets 706214 bytes 1176008605 (1.1 GB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1176008605 bytes 1176008605 (1.1 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Because these virtual machines are created on the same system, they default to the same MAC address which assigns the same IP address across all of them. To properly simulate pinging, I changed the network settings in the OracleVM hypervisor, changed the network adapter to "Bridged Adapter", then I temporarily modified their DHCP addresses to static and changed it to the following:

1. Workstation address (same): IPv4: 192.168.56.15; Gateway: 192.68.56.1 (Virtual Box Host-only address)

Cancel Wired Apply

Details Identity **IPv4** IPv6 Security

IPv4 Method

☐ Automatic (DHCP) ☐ Link-Local Only

☒ **Manual** ☐ Disable

☐ Shared to other computers

Addresses

Address	Netmask	Gateway	
192.168.56.15	255.255.255.0	192.168.56.1	

2. Server1 Address: 192.168.56.16; Gateway: 192.68.56.1 (Virtual Box Host-only address)

Addresses

Address	Netmask	Gateway	
192.168.56.16	255.255.255.0	192.168.56.1	

DNS Automatic ☒

3. Server2 Address 192.168.56.17; Gateway: 192.68.56.1 (Virtual Box Host-only address)

Addresses

Address	Netmask	Gateway	
192.168.56.17	255.255.255.0	192.168.56.1	

2. Make sure that they can ping each other.

- 2.1 Connectivity test for Local Machine 1 to Server 1: ☒ Successful ☐ Not Successful

```
santiago@workstation:~$ ping 192.168.56.16 -c4
PING 192.168.56.16 (192.168.56.16) 56(84) bytes of data:
64 bytes from 192.168.56.16: icmp_seq=1 ttl=64 time=0.557 ms
64 bytes from 192.168.56.16: icmp_seq=2 ttl=64 time=0.590 ms
64 bytes from 192.168.56.16: icmp_seq=3 ttl=64 time=0.521 ms
64 bytes from 192.168.56.16: icmp_seq=4 ttl=64 time=0.453 ms
```

- 2.2 Connectivity test for Local Machine 1 to Server 2: ☒ Successful ☐ Not Successful

```
santiago@workstation:~$ ping 192.168.56.17 -c4
PING 192.168.56.17 (192.168.56.17) 56(84) bytes of data.
64 bytes from 192.168.56.17: icmp_seq=1 ttl=64 time=0.999 ms
64 bytes from 192.168.56.17: icmp_seq=2 ttl=64 time=0.837 ms
64 bytes from 192.168.56.17: icmp_seq=3 ttl=64 time=0.628 ms
64 bytes from 192.168.56.17: icmp_seq=4 ttl=64 time=0.901 ms
```

2.3 Connectivity test for Server 1 to Server 2: ☒ Successful ☐ Not Successful

```
server1@server1:~$ ping 192.168.56.17 -c4
PING 192.168.56.17 (192.168.56.17) 56(84) bytes of data.
64 bytes from 192.168.56.17: icmp_seq=1 ttl=64 time=1.82 ms
64 bytes from 192.168.56.17: icmp_seq=2 ttl=64 time=0.499 ms
64 bytes from 192.168.56.17: icmp_seq=3 ttl=64 time=0.588 ms
64 bytes from 192.168.56.17: icmp_seq=4 ttl=64 time=0.779 ms

--- 192.168.56.17 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.499/0.920/1.817/0.527 ms
server1@server1:~$
```

Changing their IP addresses allowed a successful ping across all devices. Otherwise, these machines would be pinging themselves since they shared the same 10.0.2.15 IP address using DHCP.

Task 4: Verify SSH connectivity on Server 1, Server 2, and Local Machine.

1. On the Local Machine, issue the following commands:

1.1 `ssh username@ip_address_server1` for example, `ssh jvtaylor@192.168.56.120`

1.2 Enter the password for server 1 when prompted

```
santiago@workstation:~$ ssh server1@192.168.56.16
The authenticity of host '192.168.56.16 (192.168.56.16)' can't be established.
ED25519 key fingerprint is: SHA256:2TegAK+710YBgw+oM5bY3Fq4NqhaxwrkDUR5KwpCbY0
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.56.16' (ED25519) to the list of known hosts.
server1@192.168.56.16's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-27-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro
```

1.3 Verify that you are on server 1. The user should be in this format `user@server1`. For example, `jvtaylor@server1`

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.

```
server1@server1:~$ SSH CONNECTION ESTABLISHED
```

2. Logout of Server 1 by issuing the command *control + D*.

```
server1@server1:~$  
logout  
Connection to 192.168.56.16 closed.  
santiago@workstation:~$
```

3. Do the same for Server 2.

```
santiago@workstation:~$ ssh server2@192.168.56.17  
The authenticity of host '192.168.56.17 (192.168.56.17)' can't be established.  
ED25519 key fingerprint is: SHA256:2rb1cPJSLUsTX6s7cSQvuZnHtVcfl0Ykv9MXcXyikfI  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.168.56.17' (ED25519) to the list of known hosts.  
server2@192.168.56.17's password:  
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-27-generic x86_64)
```

```
server2@server2:~$  
logout  
Connection to 192.168.56.17 closed.  
santiago@workstation:~$
```

4. Edit the hosts of the Local Machine by issuing the command *sudo nano /etc/hosts*. Below all texts type the following:
 - 4.1 *IP_address server 1* (provide the ip address of server 1 followed by the hostname)
 - 4.2 *IP_address server 2* (provide the ip address of server 2 followed by the hostname)
 - 4.3 Save the file and exit.

```
santiago@workstation: ~ — sudo nano /etc/hosts  
GNU nano 8.7.1 /etc/hosts *  
# 127.0.0.1 localhost.localdomain localhost  
::1 localhost6.localdomain6 localhost6  
127.0.0.1 workstation  
192.168.56.16 server1  
192.168.56.17 server2
```

5. On the local machine, verify that you can do the SSH command but this time, use the hostname instead of typing the IP address of the servers. For example, try to do `ssh jvtaylor@server1`. Enter the password when prompted. Verify that you have entered Server 1. Do the same for Server 2.

```
santiago@workstation:~$ ssh server1@server1
The authenticity of host 'server1 (192.168.56.16)' can't be established.
ED25519 key fingerprint is: SHA256:2TegAK+710YBgw+oM5bY3Fq4NqhaxwrkDUR5KwpCbY0
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'server1' (ED25519) to the list of known hosts.
server1@server1's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-27-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your
Internet connection or proxy settings

Last login: Mon Jul 13 08:45:26 2026 from 192.168.56.15
server1@server1:~$ CONNECTION ESTABLISHED
```

```
santiago@workstation:~$ ssh server2@server2
The authenticity of host 'server2 (192.168.56.17)' can't be established.
ED25519 key fingerprint is: SHA256:2rb1cPJSLUsTX6s7cSQvuZnHtVcfl0Ykv9MXcXyikfI
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:4: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'server2' (ED25519) to the list of known hosts.
server2@server2's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-27-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

78 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release. Check your Internet connection or proxy settings

Last login: Mon Jul 13 08:53:07 2026 from 192.168.56.15
server2@server2:~$ CONNECTION ESTABLISHED
```

Mapping a host address to a hostname allows us to establish an SSH connection without manually typing the IP address. This is because SSH scans the `/etc/hosts` file to resolve the connection request, similar to how DNS queries are resolved through DNS lookups, or in this case, looking up the `/etc/hosts` file. This one of the other ways that SSH establishes a secure connection since `/etc/hosts` is a protected file of a system.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Reflections:

Answer the following:

1. How are we able to use the hostname instead of IP address in SSH commands?

When we establish an SSH connection with a hostname instead of an IP address, it scans the `/etc/hosts` file to find the IP address mapped to the hostname, similar to how DNS queries are resolved by performing DNS lookups. To do this, the `/etc/hosts` file must contain the hostname mapped to its IP address, where they are separated with a whitespace such as:

`/etc/hosts:`

192.168.56.16 server1

192.168.56.17 server2

A local machine can then establish a remote connection provided it has the hostname included in its /etc/hosts file.

2. How secured is SSH?

SSH offers top security due to its advanced encryption and authentication methods—it is currently industry standard and is widely used in most server systems and enterprises—provided that it is set up correctly. SSH supports Secure File Transfer Protocol (SFTP) and Secure Copy Protocol (SCP); files are transferred through secure network channels and are encrypted. SSH provides secure remote access because the connection channels are encrypted and can be set up using authentication keys instead of weak passwords; these methods effectively render common cyberattacks like man-in-the-middle attacks and brute force useless against ssh-based networks. As such, SSH is widely considered as the most secure networking tool and is the primary tool that system administrators use in modern server management in the industry.

References

Encryption Consulting. (n.d.). *Everything you need to know about SSH*.
<https://www.encryptionconsulting.com/everything-you-need-to-know-about-ssh/>

SSH Academy. (n.d.). *SSH (Secure Shell)*. <https://www.ssh.com/academy/ssh>

3. CONCLUSION

In conclusion, this activity is a basic recall of setting up SSH between a server and a local machine. SSH is the industry standard for cybersecurity which makes it essential for automating server management. Here, I recalled how to set up a virtual machine, set up the hosts and their addresses, and install and set up the SSH service. Moreover, I was able to test connectivity across three virtual machines, ensuring that they all can ping each other and vice versa. The only part that required more work was setting up the IP addresses of the machines. Virtual Machines are set up in a way that their MAC address is the same, so the local machine treats them as one single machine thus assigning only one IP address. I've already encountered this issue last semester so I was able to recall how I fixed it and made it work.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____



COMPUTER
ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.